TA Solution & Evaluation Guide — Lab 6

Attack on Titan · The Battle for Wall Rose · Confidential TA Document

Purpose: This reference contains the **exact Java implementation code** and test expectations for every method across all 4 classes. Use this to quickly verify student implementations and diagnose failing test cases during the in-person evaluation.

■ **Evaluation Policy: Do NOT copy-paste or write exact code on students' screens. Guide them toward the correct approach using the hints and leading questions provided below.**

Class 1 — TitanShifter.java

Stores shifter stats in wrapper types (Integer, Double, Boolean). Demonstrates parsing strings into wrappers, overloaded constructors, inner class vs. static nested class.

Constructor 1 — String Parsing: TitanShifter(String, String, String, String)

Parses intelligence strings using wrapper valueOf() methods:

```
public TitanShifter(String name, String powerStr, String heightStr, String hostileStr) {
    this.name = name;
    this.powerLevel = Integer.valueOf(powerStr);
    this.heightMeters = Double.valueOf(heightStr);
    this.isHostile = Boolean.valueOf(hostileStr);
}
```

Example Input / Call	Expected Output / Behavior
`new TitanShifter("Eren","150","15.0","true")`	`name="Eren", powerLevel=150, height=15.0, isHostile=true`
`new TitanShifter("Reiner","200","15.0","yes")`	`isHostile=false (Boolean.valueOf("yes") is false)`
`new TitanShifter("X", null, "15.0", "false")`	`throws NumberFormatException`

> ***TA Hint / Common Pitfall:*** *Students often expect 'yes' or '1' to evaluate to true. In Java, Boolean.valueOf only evaluates 'true' (case-insensitive) as true.*

Constructor 2 — Typed Constructor: TitanShifter(String, Integer, Double, Boolean)

Direct field assignment:

```
public TitanShifter(String name, Integer powerLevel, Double heightMeters, Boolean isHostile) {
    this.name = name;
    this.powerLevel = powerLevel;
    this.heightMeters = heightMeters;
    this.isHostile = isHostile;
}
```

Inner Class — TitanForm (Non-Static)

Directly accesses outer class's powerLevel field:

```
public class TitanForm {
    public static final int MAX_TRANSFORM_LIMIT = 3;
    private Integer transformCount;

    public TitanForm(String transformCountStr) {
        this.transformCount = Integer.parseInt(transformCountStr);
    }

    public TitanForm(int transformCount) {
        this.transformCount = transformCount;
    }

    public Integer getPowerSurge() {
        return powerLevel * transformCount * 2; // accesses enclosing powerLevel
    }
}
```

Example Input / Call	Expected Output / Behavior
`eren.powerLevel=100; form = eren.new TitanForm("2");`	`form.getPowerSurge() -> 100 * 2 * 2 = 400`
`new TitanForm("abc") / new TitanForm(null)`	`throws NumberFormatException`

> ***TA Hint / Common Pitfall:*** *TitanForm is non-static. Outside TitanShifter, it must be instantiated as: shifterObj.new TitanForm(...).*

Static Nested Class — CoordinatePower

Static nested class with static field and Coordinate activation logic. **Note:** Does NOT have implicit access to outer instance fields (requires explicit object reference), but can directly access static members of the enclosing class:

```
public static class CoordinatePower {
    private static Integer activeTitansControlled = 0;

    public static Integer getActiveTitansControlled() { return activeTitansControlled; }
    public static void setActiveTitansControlled(Integer count) { activeTitansControlled = count; }

    public static Boolean triggerCoordinate(TitanShifter shifter, String commandSignal) {
        int signal = Integer.parseInt(commandSignal, 2);
        if (!Boolean.TRUE.equals(shifter.getIsHostile()) && signal > 0) {
            activeTitansControlled = signal * 10;
            return true;
        }
        return false;
    }
}
```

Example Input / Call	Expected Output / Behavior
`shifter friendly, signal="1010" (10)`	`activeTitansControlled=100, returns true`
`shifter hostile, signal="1010"`	`returns false`
`shifter friendly, signal="0" (0)`	`returns false (signal not > 0)`

Provided Method — assessCombatReadiness(Integer)

Already given in skeleton for conceptual study of autoboxing/unboxing:

```
public static int[] assessCombatReadiness(Integer power) {
    power++;                                // unbox, increment, re-box
    Integer score = power + (power * 3);    // unbox, compute, re-box
    int rawScore = power + (power * 3);     // unbox, compute, primitive int
    return new int[]{score, rawScore};
}
```

Example Input / Call	Expected Output / Behavior
power = 100	power++ -> 101; score=101+303=404; rawScore=404 -> [404, 404]
power = null	throws NullPointerException on power++ unbox

Class 2 — ScoutSquad.java

Demonstrates constructor delegation using this(), varargs, array operations, and string reversal.

Constructors (Delegation & Varargs)

```
public ScoutSquad(String leader) {
    this(leader, new String[0]); // Constructor delegation
}

public ScoutSquad(String leader, String... members) {
    this.squadLeader = leader;
    this.members = members;
    this.squadPower = Integer.valueOf(this.members.length * 50 + 100);
    this.deployed = Boolean.FALSE;
}
```

Example Input / Call	Expected Output / Behavior
new ScoutSquad("Levi")	leader="Levi", members=[], power=100, deployed=false
new ScoutSquad("Levi", "Eren", "Mikasa")	members.length=2 -> power=2*50+100=200

> ***TA Hint / Common Pitfall:*** *Constructor delegation this(...) must be the very first statement in Constructor 1.*

Methods: boostSquadPower, canEngage, getFullRoster

```
public void boostSquadPower(Integer bonus) {
    this.squadPower = Integer.valueOf(this.squadPower.intValue() + bonus.intValue());
}

public boolean canEngage(TitanShifter titan) {
    if (titan == null) return false;
    return Boolean.TRUE.equals(titan.getIsHostile())
           && this.squadPower.compareTo(titan.getPowerLevel()) >= 0;
}

public String[] getFullRoster() {
    String[] roster = new String[1 + members.length];
    roster[0] = squadLeader;
    for (int i = 0; i < members.length; i++) {
        roster[i + 1] = members[i];
    }
    return roster;
}
```

Example Input / Call	Expected Output / Behavior
`squad(power=200).canEngage(titan(power=150, hostile=true))`	`returns true`
`squad(power=200).canEngage(titan(power=150, hostile=false))`	`returns false (friendly titan)`
`squad.getFullRoster() for leader="Levi", members=["Eren"]`	`["Levi", "Eren"]`

Algorithm: reverseDistressSignal(String rawSignal) [1.0 Mark in Hidden Tests]

Reverses word order, normalizes whitespace to single spaces, trims ends:

```
public static String reverseDistressSignal(String rawSignal) {
    if (rawSignal == null) return null;
    String[] words = rawSignal.trim().split("\\s+");
    String result = "";
    for (int i = words.length - 1; i >= 0; i--) {
        result = result + words[i];
        if (i > 0) {
            result = result + " ";
        }
    }
    return result.trim();
}
```

Example Input / Call	Expected Output / Behavior
`"advance towards the forest"`	`"forest the towards advance"`
`" breach at wall rose "`	`"rose wall at breach"`
`"multiple spaces between"`	`"between spaces multiple"`

> ***TA Hint / Common Pitfall:*** *Advise students to use split("\\s+") to collapse multiple contiguous spaces into clean tokens.*

Class 3 — WallDistrict.java

Covers 2D matrix traversal with null safety, radix-based integer parsing, and the Integer caching trap.

Method: calculateTotalConfirmedKills()

Nested loop summing all non-null kill entries:

```
public Integer calculateTotalConfirmedKills() {
    int total = 0;
    if (titanKillMatrix == null) return total;
    for (int i = 0; i < titanKillMatrix.length; i++) {
        if (titanKillMatrix[i] == null) continue;
        for (int j = 0; j < titanKillMatrix[i].length; j++) {
            if (titanKillMatrix[i][j] != null) {
                total += titanKillMatrix[i][j].intValue();
            }
        }
    }
    return Integer.valueOf(total);
}
```

Example Input / Call	Expected Output / Behavior
`matrix = [[3, null, 2], [1, 4, null]]`	`total = 3 + 2 + 1 + 4 = 10 (nulls skipped safely)`
`matrix = []`	`total = 0`

> ***TA Hint / Common Pitfall:*** *Remind students: adding a null Integer without checking causes a NullPointerException on auto-unboxing.*

Method: verifyTitanIdentification(String, String) ■ THE CACHING TRAP

Must compare wrapper values using .equals(), NOT ==:

```
public boolean verifyTitanIdentification(String reportId1, String reportId2) {
    Integer id1 = Integer.valueOf(reportId1);
    Integer id2 = Integer.valueOf(reportId2);
    return id1.equals(id2); // CRITICAL: .equals() compares value, == compares reference
}
```

Example Input / Call	Expected Output / Behavior
`reportId1="100", reportId2="100"`	`true (within cache range -128..127, == works by coincidence)`
`reportId1="128", reportId2="128"`	`true with .equals() | FAILS (false) if student used ==`
`reportId1="500", reportId2="500"`	`true with .equals() | FAILS (false) if student used ==`

> ***TA Hint / Common Pitfall:*** *This is the main conceptual trap of Lab 6. If a student passes Visible but fails Hidden, check for == here.*

Method: decodeSignal(String binarySignal, String hexSignal)

Parses radix-2 and radix-16 strings:

```
public int[] decodeSignal(String binarySignal, String hexSignal) {
    int urgency = Integer.parseInt(binarySignal, 2);
    int titanCount = Integer.parseInt(hexSignal, 16);
    return new int[]{urgency, titanCount};
}
```

Example Input / Call	Expected Output / Behavior
`binary="1010", hex="FF"`	`[10, 255]`
`binary="1111", hex="1A"`	`[15, 26]`
`binary="invalid"`	`throws NumberFormatException (expected)`

Class 4 — BattleOfWallRose.java (Master Orchestrator)

Coordinates the complete battle simulation, inner class instantiation, and varargs overloading.

Helper Methods & Varargs Overloading

```
// Exact-match overload
public static int deploySquadUnits(int squadSize) {
    return squadSize;
}

// Varargs overload
public static int deploySquadUnits(int firstSquad, int... otherSquads) {
    int total = firstSquad;
    for (int s : otherSquads) total += s;
    return total;
}

// Search for non-hostile titan
public TitanShifter findFriendlyShifter() {
    for (int i = 0; i < titanCount; i++) {
        if (Boolean.FALSE.equals(encounteredTitans[i].getIsHostile())) {
            return encounteredTitans[i];
        }
    }
    return null;
}
```

Example Input / Call	Expected Output / Behavior
`deploySquadUnits(5)`	`returns 5 (calls exact-match overload)`
`deploySquadUnits(5, 3, 2)`	`returns 10 (calls varargs overload)`

Master Method: executeDefenseOperation(String, String) [0.8 Marks in Hidden Tests]

Complete implementation orchestrating all 4 classes:

```
public boolean executeDefenseOperation(String commandSignal, String hexIntel) {
    this.totalHumanityDefenseScore = Integer.valueOf(0);

    for (int d = 0; d < districtCount; d++) {
        WallDistrict district = districts[d];

        // 1. Signal Intelligence
        int[] signals = district.decodeSignal(commandSignal, hexIntel);
        this.totalHumanityDefenseScore = Integer.valueOf(
            this.totalHumanityDefenseScore.intValue() + signals[0] + signals[1]
        );

        // 2. Kill Tallying
        Integer kills = district.calculateTotalConfirmedKills();
        if (kills != null) {
            this.totalHumanityDefenseScore = Integer.valueOf(
                this.totalHumanityDefenseScore.intValue() + kills.intValue()
            );
        }

        // 3. Titan Engagement
        TitanShifter spotted = district.getSpottedTitan();
        if (spotted != null) {
            if (Boolean.TRUE.equals(spotted.getIsHostile())) {
                boolean engaged = false;
                for (int s = 0; s < squadCount; s++) {
                    ScoutSquad squad = activeSquads[s];
                    if (!Boolean.TRUE.equals(squad.isDeployed()) && squad.canEngage(spotted)) {
                        squad.setDeployed(Boolean.TRUE);
                        this.totalHumanityDefenseScore = Integer.valueOf(
                            this.totalHumanityDefenseScore.intValue() + squad.getSquadPower().intValue()
                        );
                        engaged = true;
                        break;
                    }
                }
                if (!engaged) {
                    TitanShifter friendly = findFriendlyShifter();
                    if (friendly != null && squadCount > 0) {
                        // Non-static inner class instantiation syntax:
                        TitanShifter.TitanForm form = friendly.new TitanForm("2");
                        Integer surge = form.getPowerSurge();
                        activeSquads[0].boostSquadPower(surge);
                        activeSquads[0].setDeployed(Boolean.TRUE);
                        this.totalHumanityDefenseScore = Integer.valueOf(
                            this.totalHumanityDefenseScore.intValue() + activeSquads[0].getSquadPower().int
Value()
                        );
                        int[] readiness = TitanShifter.assessCombatReadiness(friendly.getPowerLevel());
```

```
                        this.totalHumanityDefenseScore = Integer.valueOf(
                            this.totalHumanityDefenseScore.intValue() + readiness[0]
                        );
                    }
                }
            } else {
                // Static nested class call:
                Boolean triggered = TitanShifter.CoordinatePower.triggerCoordinate(spotted, commandSignal);
                if (Boolean.TRUE.equals(triggered)) {
                    this.totalHumanityDefenseScore = Integer.valueOf(
                        this.totalHumanityDefenseScore.intValue() +
                        TitanShifter.CoordinatePower.getActiveTitansControlled().intValue()
                    );
                }
            }
        }
    }
    return this.totalHumanityDefenseScore.intValue() >= 100;
}
```

> ***TA Hint / Common Pitfall:*** *Watch for: (1) friendly.new TitanForm("2") syntax for non-static inner class; (2) stopping at the first undeployed squad with break; (3) resetting totalHumanityDefenseScore to 0 at the start.*

TA Quick Grading & Diagnostic Checklist

Symptom / Failing Test	Probable Cause & Where to Look in Code
testIntegerCachingBoundary128 fails	Student used id1 == id2 instead of id1.equals(id2) in WallDistrict.verifyTitanIdentification.
testTitanFormInnerClassPowerSurge fails	Inner class TitanForm did not multiply outer powerLevel * transformCount * 2 correctly.
testBooleanValueOfTrap fails	Student wrote custom boolean logic assuming 'yes' is true. Boolean.valueOf only accepts 'true'.
testReverseDistressSignalWithSpaces fails	Student used split(' ') instead of split('\\s+'), leaving empty tokens in the reversed string.
testBattleOfWallRoseHostileEngagement fails	Check inner class instantiation friendly.new TitanForm('2') and readiness score addition in executeDefenseOperation.

Reminder: All 12 Visible tests are in TestCases.jar (no console marks). All 21 Hidden tests are in HiddenTestCases.jar, which automatically computes and prints the student's score out of 10.0 Marks directly to the console when run.